CEBU INSTITUTE OF TECHNOLOGY
UNIVERSITY

IT342-G1
SYSTEMS INTEGRATION AND ARCHITECTURE 1

FUNCTIONAL REQUIREMENTS SPECIFICATION (FRS)

Project Title: User Registration and Authentication

Prepared By: Tiongzon, Jay Yan C.

Date of Submission: 02/14/2026

Version: 3

Table of Contents

1. Introduction....3
   1.1. Purpose....3
   1.2. Scope....3
   1.3. Definitions, Acronyms, and Abbreviations....3
2. Overall Description....3
   2.1. System Perspective....3
   2.2. User Classes and Characteristics....3
   2.3. Operating Environment....3
   2.4. Assumptions and Dependencies....3
3. System Features and Functional Requirements....3
   3.1. Feature 1:....3
   3.2. Feature 2:....3
4. Non-Functional Requirements....3
5. System Models (Diagrams)....4
   5.1. ERD....4
   5.2. Use Case Diagram....4
   5.3. Activity Diagram....4
   5.4. Class Diagram....4
   5.5. Sequence Diagram....4
6. Appendices....4

1. Introduction

1.1. Purpose

This system provides a secure gateway for users to access the application by managing their credentials and session states. It is intended for developers, project stakeholders, and system administrators.

1.2. Scope

The system will handle Sign-ups, Login, Logout, and Token Management. It serves as the foundational security layer for the application, ensuring that only authenticated users can access protected routes.

1.3. Definitions, Acronyms, and Abbreviations

- FRS – Functional Requirements Specification: A document that describes how a system must behave and its specific features.
- UI – User Interface: The space where interactions between humans and the application occur.
- API – Application Programming Interface: A set of rules that allow the React frontend and Spring Boot backend to communicate.
- ERD – Entity Relationship Diagram: A visual representation of the database structure and how data entities relate to one another.
- JWT – JSON Web Token: A secure method for transmitting information between parties as a JSON object, used here for authentication tokens.
- BCrypt: A password-hashing function used to securely store credentials in the database.
- ReactJS: A JavaScript library used for building the user interface and handling the frontend logic.
- Spring Boot: A Java-based framework used to create the REST API and handle the backend business logic.

2. Overall Description

2.1. System Perspective

This system operates as a standalone web application. It consists of a ReactJS frontend interface and a Spring Boot backend API that manages data persistence in a MySQL database. It handles its own user authentication using JWTs.

2.2. User Classes and Characteristics

- Guest (Unregistered): Can view the landing page and access the registration or login forms.
- Registered User: Can log in using valid credentials, receive an authentication token, and access the protected dashboard.

2.3. Operating Environment

- Client-side: Any modern web browser (Chrome, Firefox, Safari, Edge).

- Server-side: A web server running the Spring Boot application.
- Database: MySQL.
- Network: Stable internet connection required for all operations.

2.4. Assumptions and Dependencies

- It is assumed that users will provide accurate and honest information regarding lost or found items.
- The system depends on a reliable hosting provider to ensure 24/7 availability.
- Users must have a valid email address for account registration and recovery.

3. System Features and Functional Requirements

3.1. Feature 1: Account Creation (Registration)

Description: Enables new users to create an account by providing valid personal information and credentials.

Functional Requirements:

- The system shall enable guest users to register using a unique username, email address, and a secure password.
- The system shall validate that the email address is in a correct format and not already in use.
- The system shall encrypt the user's password using **BCrypt** before storing it in the MySQL database.
- The system shall create a new user record with a default role (e.g., "USER") upon successful validation.

3.2. Feature 2: User Authentication and Session Management

Description: Enables registered users to log in to the system, maintain a secure session, and safely log out.

Functional Requirements:

- The system shall verify user identity by comparing the provided password with the hashed password stored in the database.
- The system shall generate and issue a signed **JWT (JSON Web Token)** upon a successful login.
- The system shall require the frontend to include the valid JWT in the header of subsequent API requests to access protected resources (e.g., the Dashboard).
- The system shall provide a logout function that clears the stored token from the client, effectively terminating the session.

4. Non-Functional Requirements

- **Security:** Passwords must never be stored in plain text; they must be hashed using BCrypt. API endpoints must be protected to ensure only requests with valid tokens are processed.

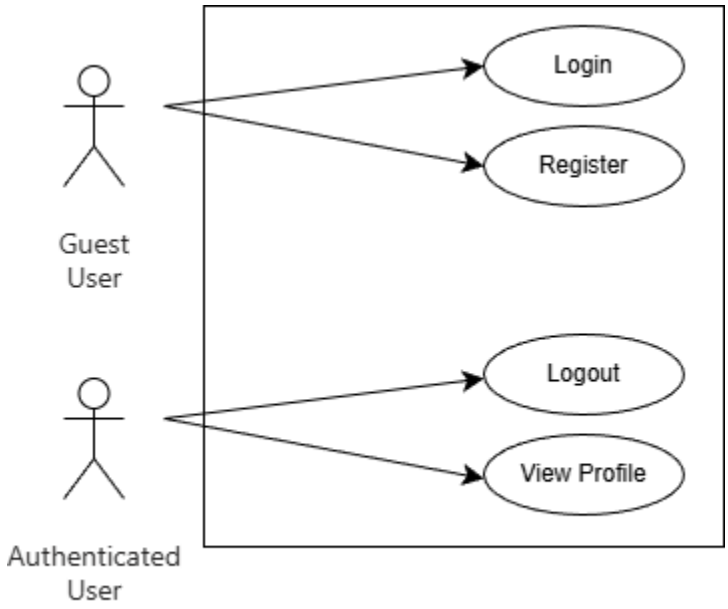
- **Performance:** The login and registration processes should complete within 2 seconds to ensure a responsive user experience.
- **Usability:** The UI should provide clear error messages (e.g., "Invalid username or password") to guide the user in case of mistakes.
- **Reliability:** The system handles database connection failures gracefully by returning appropriate HTTP error codes (e.g., 500 Internal Server Error) instead of crashing.
- **Availability:** The service is designed to be available 24/7, dependent on the hosting server's uptime.

5. System Models (Diagrams)

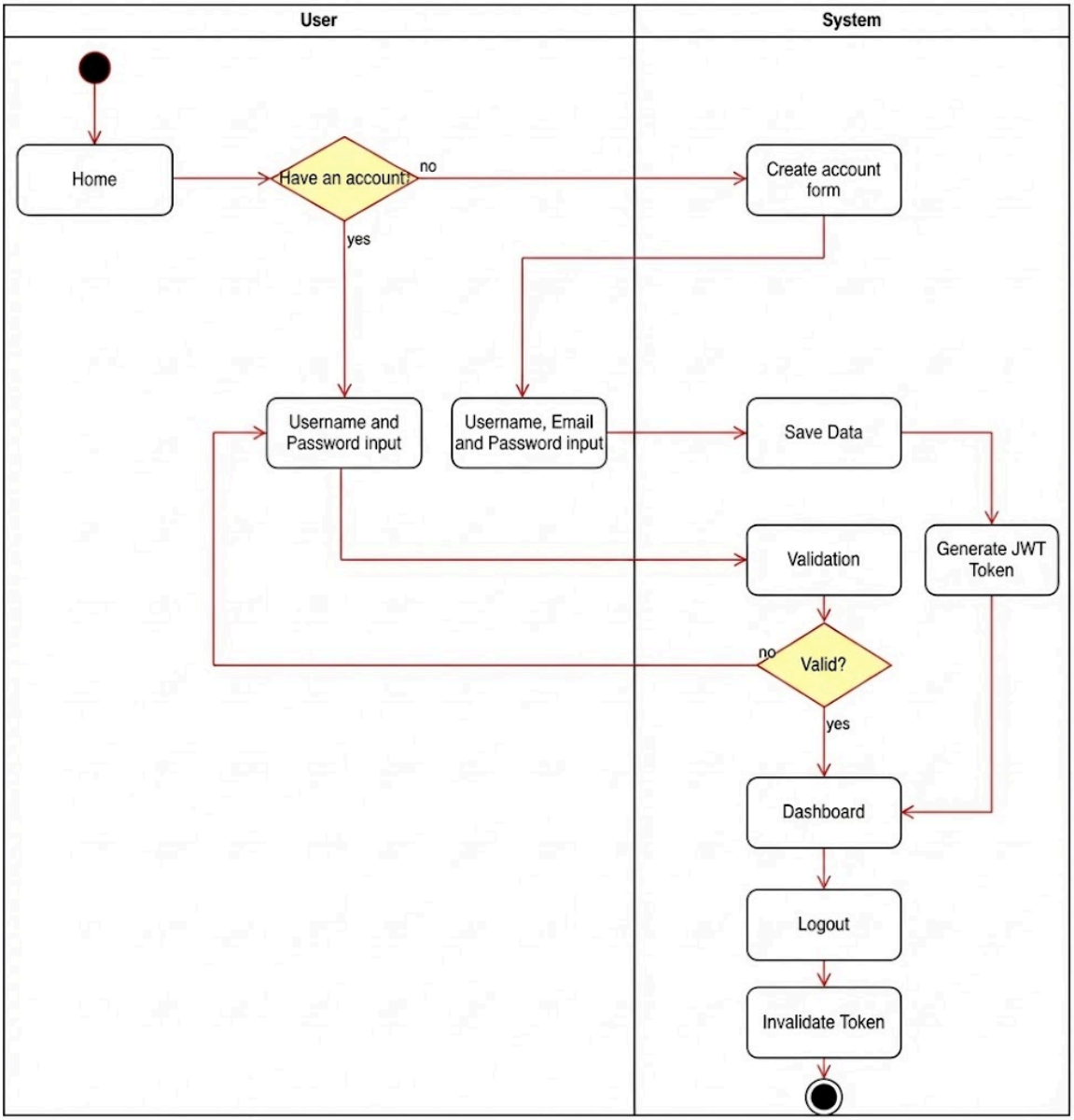
5.1. ERD

users	
PK	<u>user_id</u>
	username
	email
	password
	avatar_url
	role (user, admin)
	is_active
	created_at
	last_login

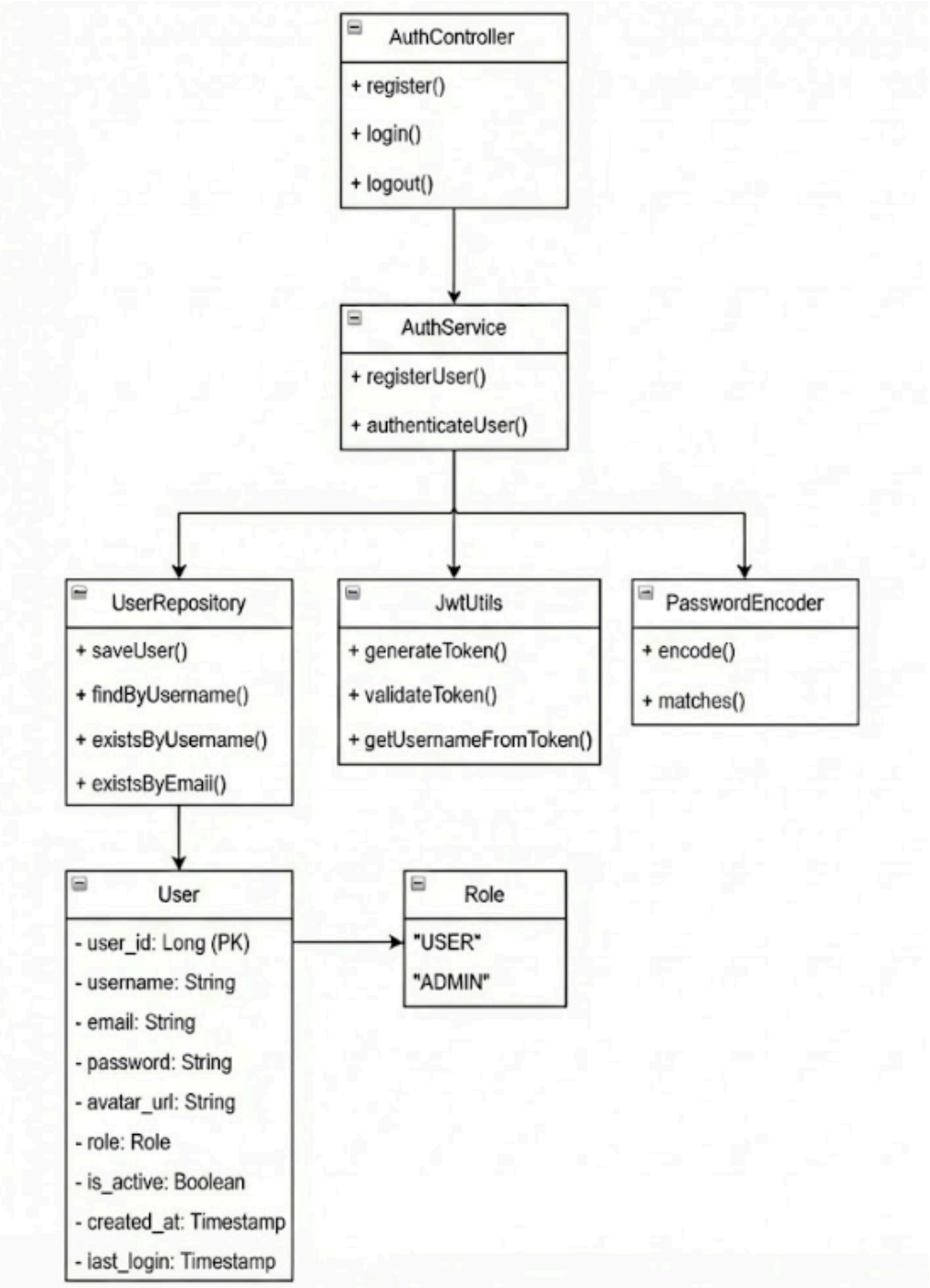
5.2. Use Case Diagram



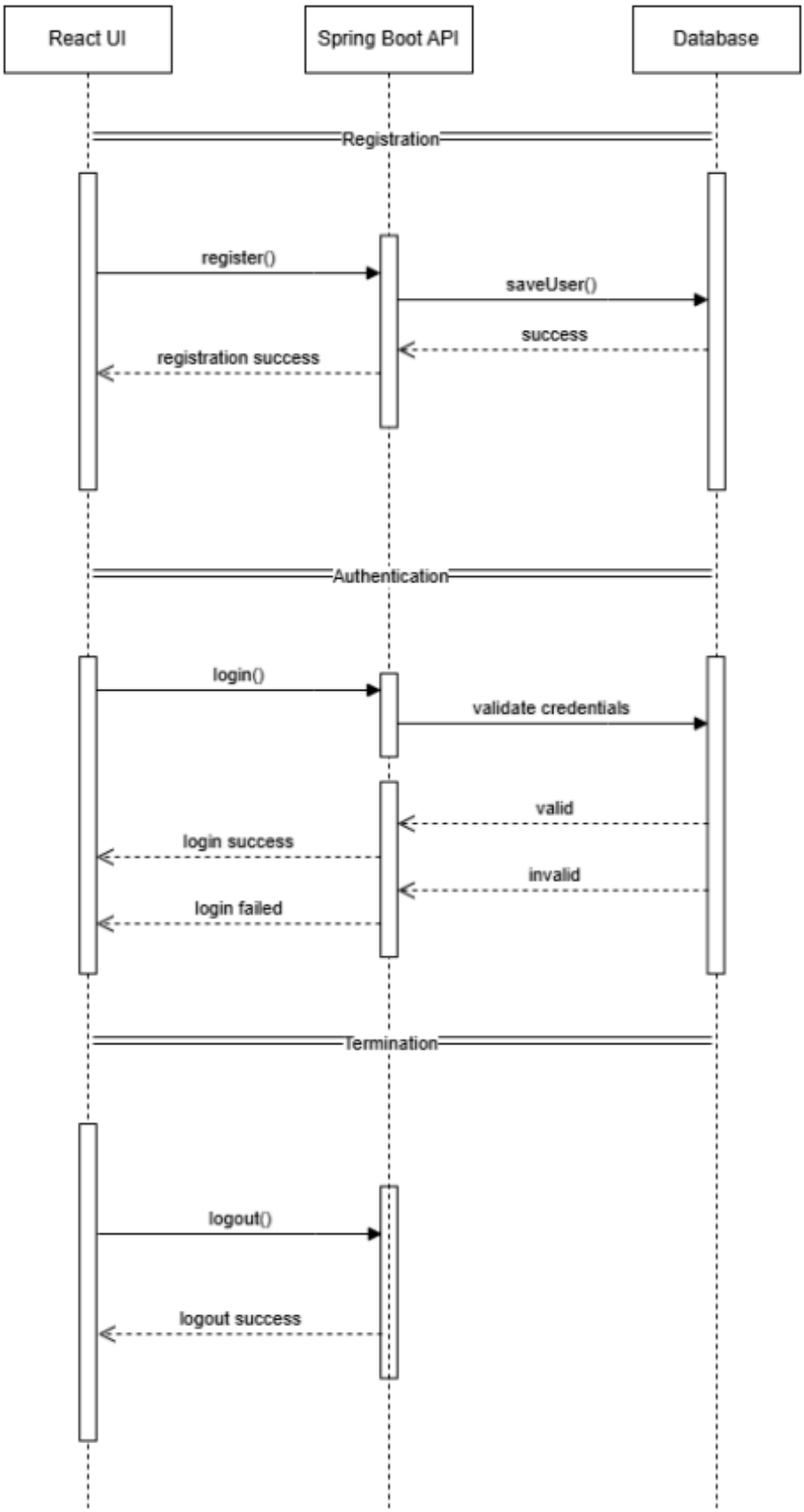
5.3. Activity Diagram



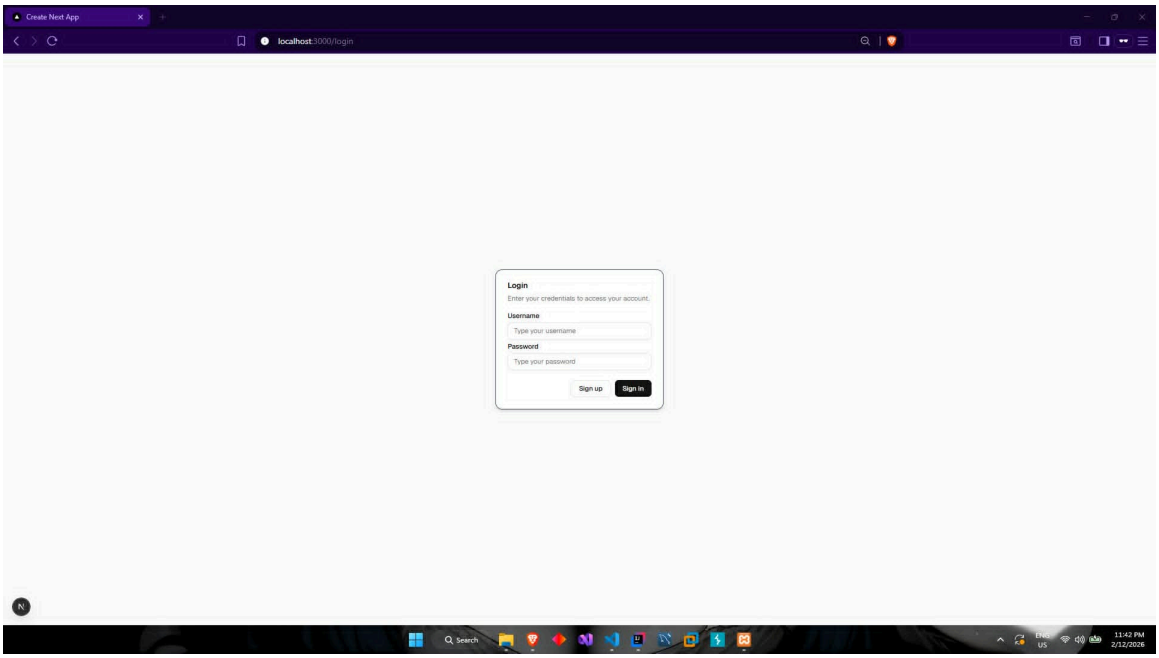
5.4. Class Diagram

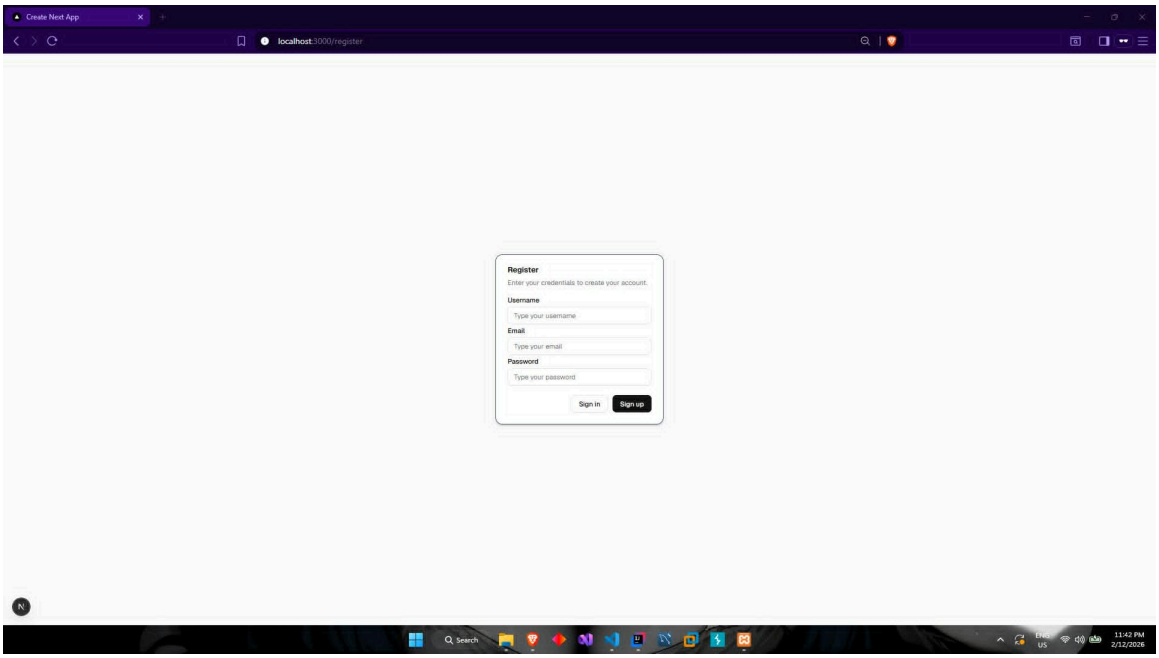


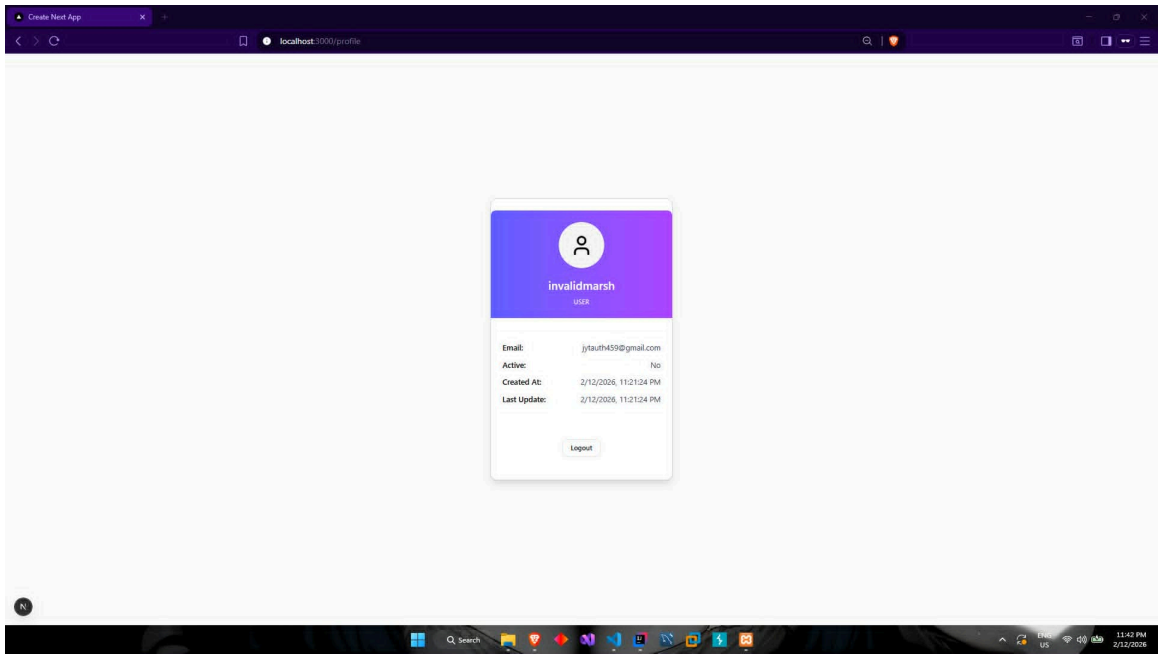
5.5. Sequence Diagram

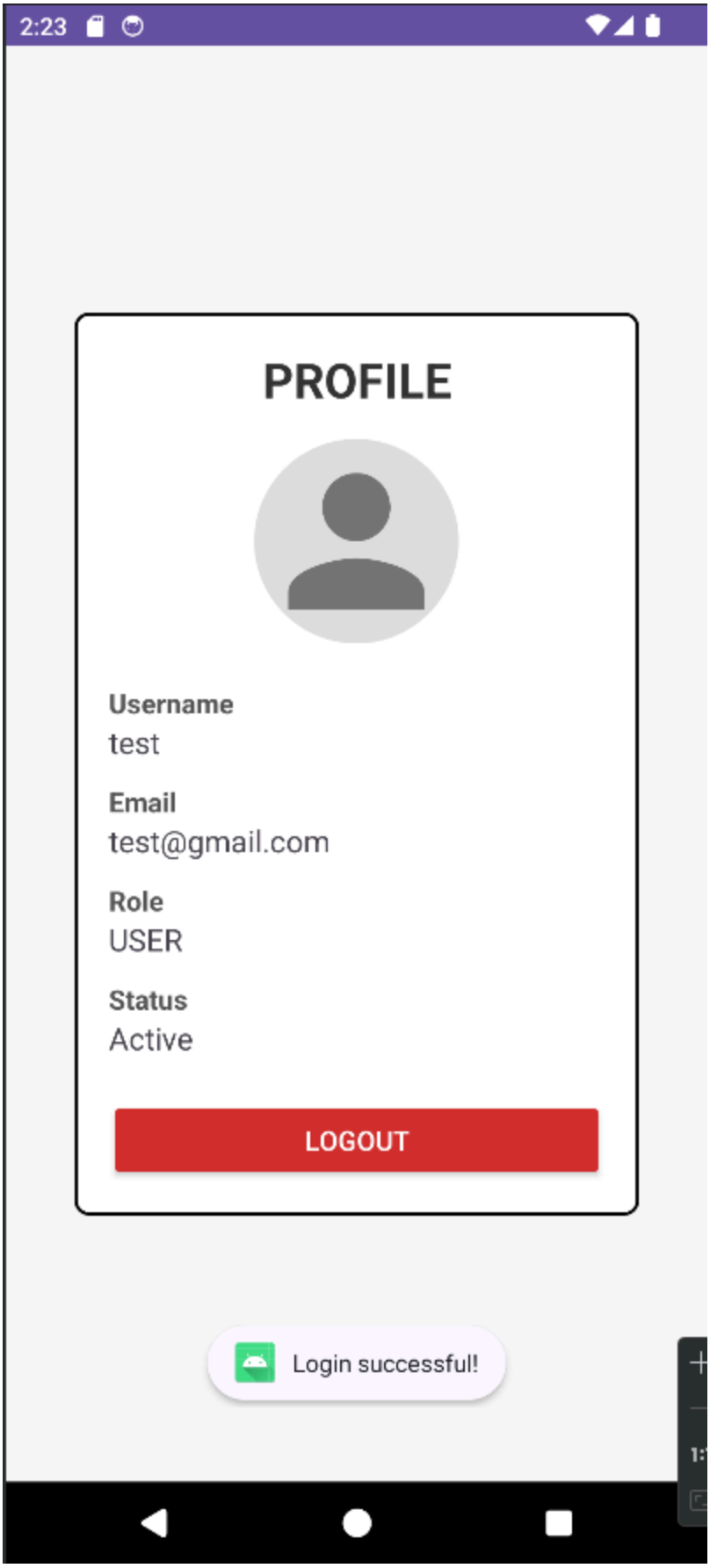

React UI
Spring Boot API
Database
Registration
register()
saveUser()
success
registration success
Authentication
login()
validate credentials
valid
login success
invalid
login failed
Termination
logout()
logout success

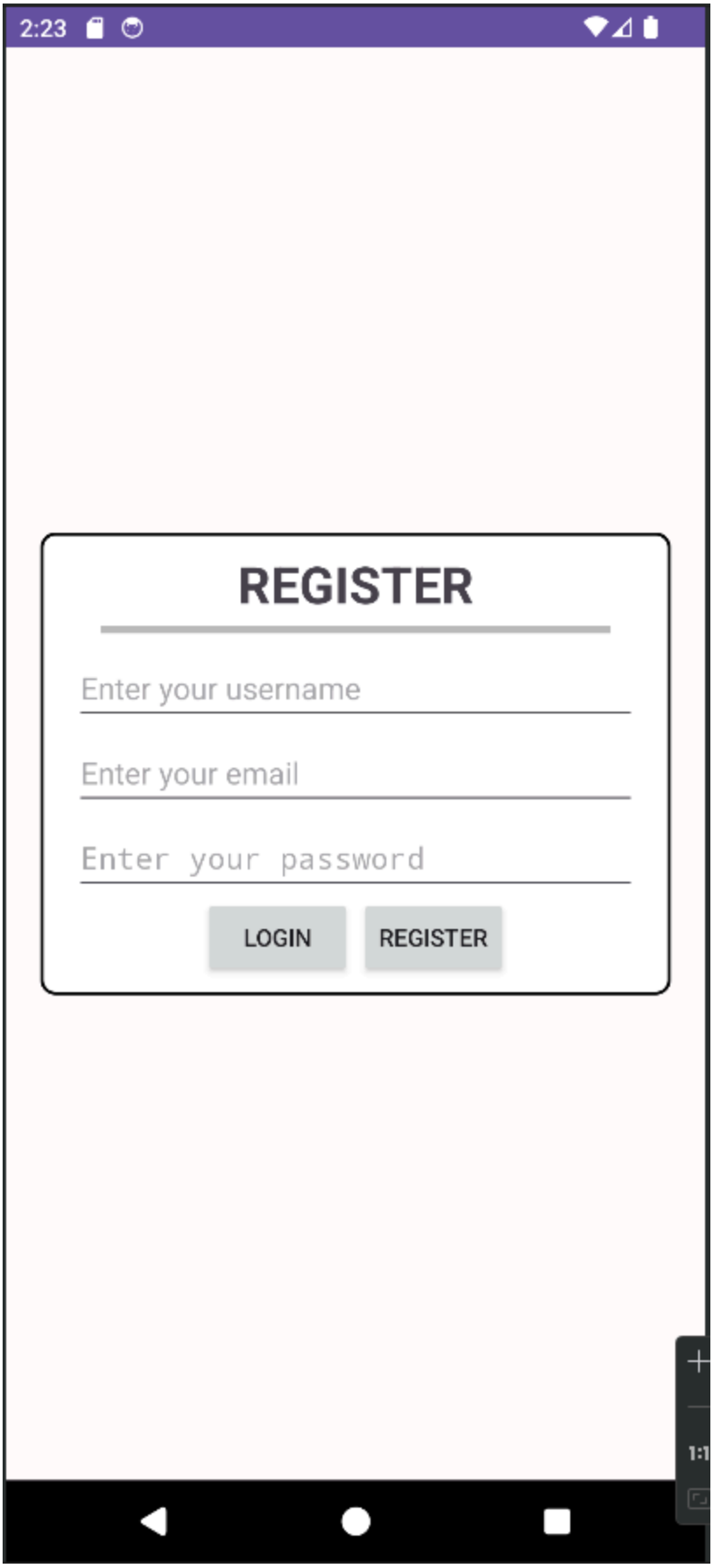
5.6. Screenshots

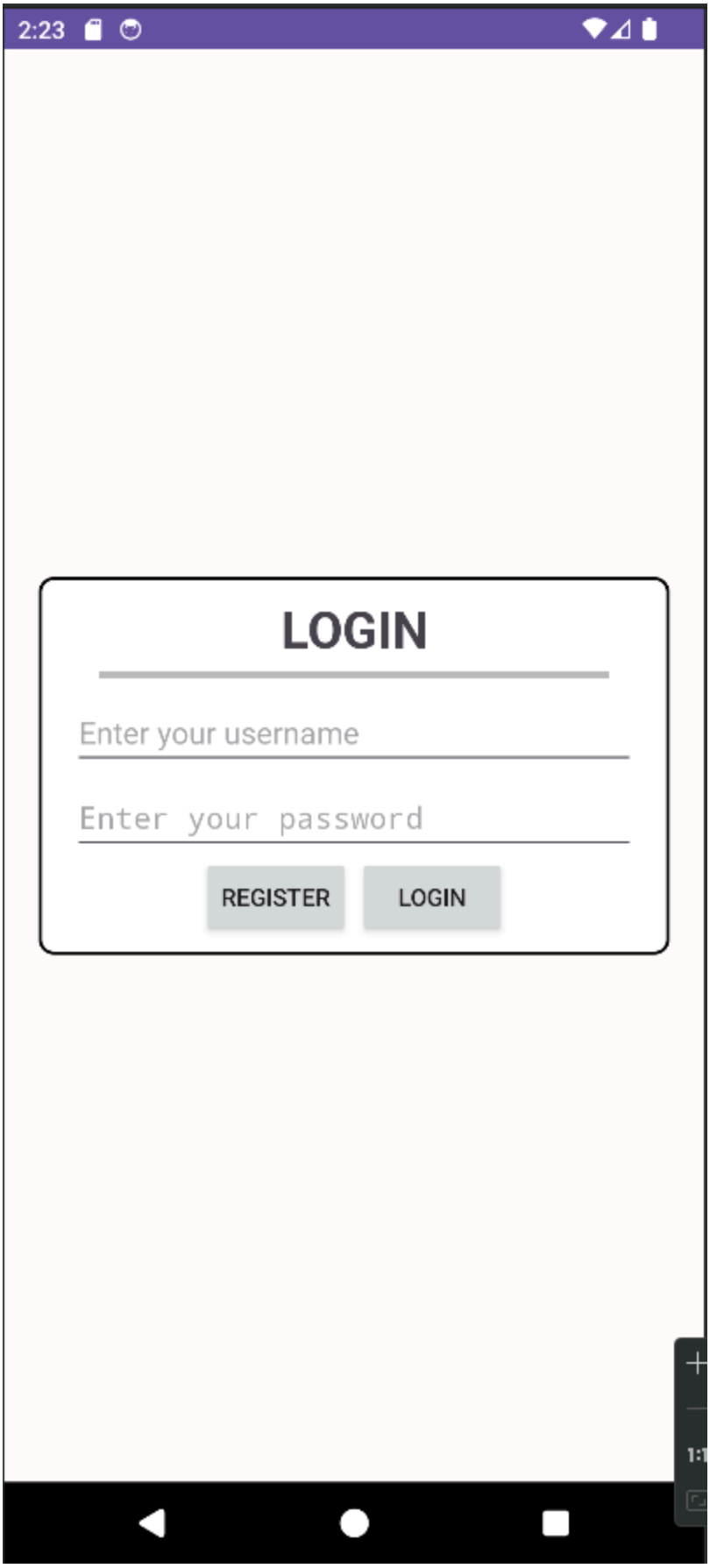





Create Next App
localhost:3000/profile
invalidmarsh
USER
Email:
jytauth459@gmail.com
Active:
No
Created At:
2/12/2026, 11:21:24 PM
Last Update:
2/12/2026, 11:21:24 PM
Logout
Search
ENG
US
11:42 PM
2/12/2026


2:23
PROFILE
Username
test
Email
test@gmail.com
Role
USER
Status
Active
LOGOUT
Login successful!


2:23
REGISTER
Enter your username
Enter your email
Enter your password
LOGIN
REGISTER


2:23
LOGIN
Enter your username
Enter your password
REGISTER
LOGIN

6. Appendices

- Technological References: Developed using Next.js for the UI, Spring Boot for the API, and MySQL for data storage.
- Security: Uses BCrypt for password hashing and JSON Web Tokens (JWT) for secure user sessions.
- Tools: All diagrams were created using draw.io / diagrams.net.
- External Services: Assumes access to a stable internet connection for all operations.